Basic Matrix Multiplication in Python, Java and C: How Matrix Size Drives Runtime and Memory

Abdelaziz Choukri Mohamed · Big Data 2026–2027, Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria

Individual Assignment 1 — Basic Matrix Multiplication in Different Languages · 2026-10-17
Code, raw data and scripts: https://github.com/abdela758/bigdata-matmul

Abstract. This study measures how the matrix size n affects the runtime and the memory of the textbook triple-loop multiplication of dense $n \times n$ `double` matrices in Python, Java and C, on one Windows laptop (Intel Core i5-1135G7), with identical input files, loop order and timing protocol. Sizes were increased until a single multiplication exceeded 120 s: the largest size that met the budget was $n = 1000$ for Python, 2000 for Java and 2500 for C. All three implementations return bit-identical results. Kernel time (median and interquartile range over repeated batches and up to three launches) grows roughly, but not exactly, as n^3: the time per inner iteration, t/n^3, is nearly constant for small n and then rises in every language, steeply in C and Java above $n \approx 1000$–1500. Python is about 73× slower than C (range 67–78× for $n \geq 100$); Java is 1.2–1.7× slower than C up to $n = 1000$ but 4.9× slower at $n = 1500$, so in this study it does not approach C at large sizes. At $n = 1000$ the processes peak at 127 MiB (Python), 66 MiB (Java) and 27 MiB (C), on top of runtime baselines of 11, 42 and 4 MiB. The conclusions hold for these implementations, settings, inputs and this machine; explanations of the slowdown at large sizes are hypotheses, not measurements.

1. Introduction and objective

Multiplying two dense $n \times n$ matrices with three nested loops needs n^3 multiply-add operations and is the standard starting point for talking about performance and scalability [1]. The algorithm is the same in every language, but the cost of each operation is not: an interpreter, a just-in-time (JIT) compiler and an ahead-of-time compiler turn the same source into very different machine behaviour, and the way each language lays a matrix out in memory decides how well the processor caches can help [2].

Research question. *How does the matrix size n affect the runtime and the memory use of the textbook triple-loop multiplication of dense* `double` *matrices implemented in Python, Java and C, under one fixed and documented set of conditions on one machine?* The factor that is changed is n (and, as a second factor, the language); the outcomes are the time of one multiplication and the peak memory of the process.

Expectations. They were written down from the algorithmic model and the course material before the final measurements were analysed, but after a small pilot run (used to choose sizes and limits) had already shown that large sizes cost more per operation, so they are informed expectations, not blind ones.

1. **E1 – cubic growth.** The time of one multiplication grows like n^3, so the time per inner iteration, t/n^3, should be roughly constant, at least for small sizes.
2. **E2 – constants differ by language.** The constant should be far larger in Python than in Java or C; Java, once the JIT has compiled the loop, should be in the same order of magnitude as C.
3. **E3 – memory effects.** In the *i-j-k* order the inner loop walks matrix B down a column, which uses the cache poorly [2]. Once B ($8n^2$ bytes) outgrows the 1.25 MiB L2 cache ($n \approx 400$) and then the 8 MiB L3 cache ($n \approx 1000$), t/n^3 may rise.
4. **E4 – memory.** Every process has a fixed runtime baseline, and on top of it the data grow with n^2; Python, which stores each number as a separate object, should need much more per element than C.

2. Background

Algorithm. For square matrices, $C_{ij} = \Sigma_k A_{ik} B_{kj}$; evaluating it directly takes $\Theta(n^3)$ operations [1]. In the loop order *i-j-k* used here, the inner loop reads A along a row (consecutive addresses) and B along a column (addresses n elements apart), the access pattern analysed by Drepper [2, Sec. 6.2.1].

The three languages. CPython runs bytecode in an interpreter and stores every number as an object, so a list of floats holds pointers to separate 24-byte `float` objects. HotSpot (Java) starts in an interpreter, profiles the code and recompiles hot loops with its JIT compilers; a `double[][]` is an array of row arrays of unboxed 8-byte values. A C compiler produces native code before the program runs and a `malloc`ed block stores the elements back to back. GCC enables loop interchange at `-O3` [4], which could silently change the loop order, so the main comparison uses `-O2`. All three programs use IEEE-754 double precision and the same operation order.

3. Methodology

3.1 Implementations

The straightforward triple loop with $O(n^3)$ work is implemented in all three languages; no optimised library routine replaces the loops (NumPy is used only to check results). The loops are, literally:

Python
```
for i in range(n):
    for j in range(n):
        s = 0.0
        for k in range(n):
            s += a[i][k] * b[k][j]
        c[i][j] = s
```

Java
```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        double sum = 0.0;
        for (int k = 0; k < n; k++) sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
```

C
```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        double sum = 0.0;
        for (int k = 0; k < n; k++) sum += a[(size_t)i*n + k] * b[(size_t)k*n + j];
        c[(size_t)i*n + j] = sum;
    }
```

Table 1. The three implementations. Same in all three: *i-j-k* loop order, `double`, local accumulator, one kernel thread, same input files, same timing rules.

	Python	Java	C
Runtime / compiler	CPython 3.12.0 (python.org build), standard library only	openjdk version 25.0.2 2026-01-20 (HotSpot, tiered JIT, G1 GC)	GCC (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r2) 16.2.0
Flags / settings	default interpreter	`-Xmx4g`; other JVM options default	`-std=c11 -Wall -Wextra -O2`
Matrix storage	list of *n* row lists; each element points to a 24-byte `float` object	`double[n][n]`: *n* row arrays of unboxed 8-byte values	one `double[n*n]` per matrix, row-major, 8 bytes per element
Kernel timer	`time.perf_counter_ns`	`System.nanoTime`	`QueryPerformanceCounter`
Sizes *n* planned	10, 20, 50, 100, 200, 300, 500, 700, 1000, 1500	10, 20, 50, 100, 200, 300, 500, 700, 1000, 1500, 2000, 2500	10, 20, 50, 100, 200, 300, 500, 700, 1000, 1500, 2000, 2500, 3000

The accumulator is a local variable in all three languages, so each performs the same additions in the same order. The Python loop is deliberately the unoptimised textbook version. The unavoidable differences are the storage layouts in Table 1, the fact that Python allocates one new `float` object per result element inside the timed kernel, and that the JVM compiles code at run time. Storage layout and compiler flags are not varied in this study.

3.2 Inputs

All implementations read the same files `A_n.bin` and `B_n.bin` (binary double precision, row-major, values uniform in [0, 1)), generated with NumPy's `default_rng([20261012, n, which])`. Each file is reproducible on its own, and its SHA-256 hash is stored in `inputs/checksums.json`, so every language multiplies exactly the same matrices.

3.3 Environment

Table 2. Hardware and software environment (`results/environment.json`).

Item	Value
Machine	HP 250 G8 Notebook PC, Windows 11 Pro (build 26200)
CPU	11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz: 4 physical cores, 8 logical processors; L2 1.25 MiB per core, L3 8 MiB shared
RAM / power	15.7 GB; AC power, Windows "Balanced" power plan, no changes to frequency or affinity settings
Threads	The kernel runs on one thread in every language (no parallel code, no numerical library); the JVM also runs its own JIT and GC threads.

3.4 Correctness validation

Before any benchmark, `scripts/run_tests.py` multiplies test matrices with each implementation: identity and zero matrices, hand-computed 2 × 2 and 3 × 3 products, and random integer matrices stored as doubles (exact in any summation order) must match **bit for bit**. Random real matrices (uniform in [0, 1) and [−1, 1)) and the shared benchmark inputs are compared with NumPy's `A @ B` using an absolute and a relative tolerance of 10^{-9} (`numpy.allclose`). Double precision carries about 16 significant digits and summing a few hundred products only changes the last digits, so 10^{-9} is strict enough to catch any real error (a wrong index gives errors of order 1) while allowing the different summation order of NumPy's library. Finally, the three implementations must return **bit-identical** matrices. The sum of all elements of C is also printed after every timed batch, which shows that the computation ran and lets results be compared across languages.

3.5 Timing and memory protocol

- **What is timed.** The multiplication kernel only. Input files are read, and the result matrix allocated, before timing; the kernel assigns every element of C, so no initialisation is needed or timed. No end-to-end time is mixed with this kernel time.
- **Harness.** The course material suggests JMH, pytest-benchmark and `perf` [3, 8]. Here one small harness with the same rules is used for all three languages, so that the timing boundary and the memory measurement are identical everywhere (`perf` is not available on Windows). It was not cross-checked against JMH. Timers are monotonic; in CPython on Windows `perf_counter` is backed by `QueryPerformanceCounter` (resolution 100 ns here) [5], the same source as the C program [6].
- **Batches.** To keep every timed interval much longer than the timer resolution, a *batch* runs m multiplications back to back and the program reports time per multiplication; m is found by doubling until a batch lasts at least 0.2 s (m = 1 when one multiplication already takes longer).
- **Warm-up and repetitions.** The calibration batches plus 0–5 further warm-up batches (`config/experiment.json`) are run first and never enter a statistic; then 5–15 *measured* batches. Each configuration is run in 1–3 separate processes (*launches*), interleaved in time; cheap configurations get three launches, the expensive ones one or two (Appendix A).
- **Statistics.** All measured batches of all successful launches of a configuration are pooled. The central value is the **median**; variability is the **interquartile range** (IQR), shown as [Q1–Q3]. All raw measurements are kept in `results/raw_samples.csv`.
- **Memory.** The *peak working set* of the whole process (`PeakWorkingSetSize` [7]), read by the driver after the process ends; one value per launch. It includes the language runtime, so it is not the same as array size or Java heap use, and the figures must not be compared as if they were. The runtime baseline is estimated from the smallest size (n = 10).

3.6 Sizes and budgets

Sizes grow roughly geometrically (10 … 3000, where feasible) and avoid powers of two. The budgets were declared before the final run: **at most 120 s per single multiplication** (a launch that prints nothing for that long is stopped and recorded as a timeout), **30 min per launch** and **4 GiB of memory**. The largest size that met the budget is reported separately from the largest attempted, and timeouts are reported at their limit, never with an invented time. Three $n \times n$ double matrices need $24n^2$ bytes (216 MB at $n = 3000$), so time, not memory, is the limit that binds.

4. Results

4.1 Correctness

All 84 checks passed (Table 3). The largest difference between any implementation and NumPy on random matrices was 4.3e-14, more than four orders of magnitude below the 10^{-9} tolerance, so the tolerance would catch a real error but does not hide a marginal one. The sum of all elements of C printed after every timed batch was identical in all three languages for every size (`results/tables/checksums.csv`).

Table 3. Correctness checks (each row counts the three implementations × test cases).

Check	Sizes n	Criterion	Passed	Largest difference to NumPy
I × X = X	1, 7, 33	bit-exact	9/9	–
X × I = X	1, 7, 33	bit-exact	9/9	–
0 × X = 0	1, 7, 33	bit-exact	9/9	–
hand-computed 2 × 2 product	2	bit-exact	3/3	–
hand-computed 3 × 3 product	3	bit-exact	3/3	–
random integers 0..9 vs NumPy int64 product	50, 100	bit-exact	6/6	–
random reals in [0, 1) vs NumPy	10, 100, 200	|C − NumPy| ≤ 1e-9 + 1e-9·|NumPy|	9/9	3.6e-14
random reals in [−1, 1) vs NumPy	10, 100, 200	|C − NumPy| ≤ 1e-9 + 1e-9·|NumPy|	9/9	1.1e-14
the shared benchmark inputs vs NumPy	100, 300	|C − NumPy| ≤ 1e-9 + 1e-9·|NumPy|	6/6	4.3e-14
C, Java and Python results identical	1, 2, 3, 7, 10, 33, 50, 100, 200, 300	bit-identical	21/21	–
Total			**84/84**	

4.2 Runtime

Table 4 and Figure 1 give the time of one multiplication. On the log-log plot the three curves are nearly parallel straight lines from $n = 10$ to $n \approx 1000$: one multiplication goes from 67.6 µs (Python), 930 ns (Java) and 537 ns (C) at $n = 10$ to 92.4 s, 2.38 s and 1.37 s at $n = 1000$. Above that, the Java and C curves bend upwards (Java first); Python stops at $n = 1000$ because the next size, 1500, exceeded the 120 s limit.

Table 4. Time of one multiplication: median [Q1–Q3] over all measured batches of all launches (counts in Appendix A).

n	Python	Java	C
10	67.6 µs [63.3 µs–74.3 µs]	930 ns [869 ns–976 ns]	537 ns [504 ns–565 ns]
20	509 µs [484 µs–527 µs]	7.09 µs [6.73 µs–7.74 µs]	4.34 µs [4.14 µs–4.59 µs]
50	7.82 ms [7.16 ms–8.72 ms]	105 µs [102 µs–108 µs]	85.8 µs [81.5 µs–92.1 µs]
100	62.5 ms [58.7 ms–65.2 ms]	965 µs [935 µs–1.03 ms]	802 µs [757 µs–975 µs]
200	526 ms [507 ms–563 ms]	8.6 ms [8.35 ms–8.89 ms]	7.16 ms [6.94 ms–7.62 ms]
300	1.91 s [1.85 s–2 s]	31 ms [30 ms–32.7 ms]	26.3 ms [25.3 ms–27.9 ms]
500	9.86 s [9.63 s–10.1 s]	200 ms [189 ms–206 ms]	133 ms [130 ms–141 ms]
700	29.3 s [29.1 s–30 s]	618 ms [589 ms–693 ms]	408 ms [387 ms–534 ms]
1000	92.4 s [92.2 s–92.9 s]	2.38 s [2.32 s–2.48 s]	1.37 s [1.35 s–1.39 s]
1500	not run / timeout	26.8 s [26.8 s–26.8 s]	5.43 s [5.34 s–6.64 s]
2000	not run / timeout	82.3 s [81.9 s–82.8 s]	28.4 s [27.2 s–30.2 s]
2500	not run / timeout	not run / timeout	68.8 s [68.4 s–68.9 s]

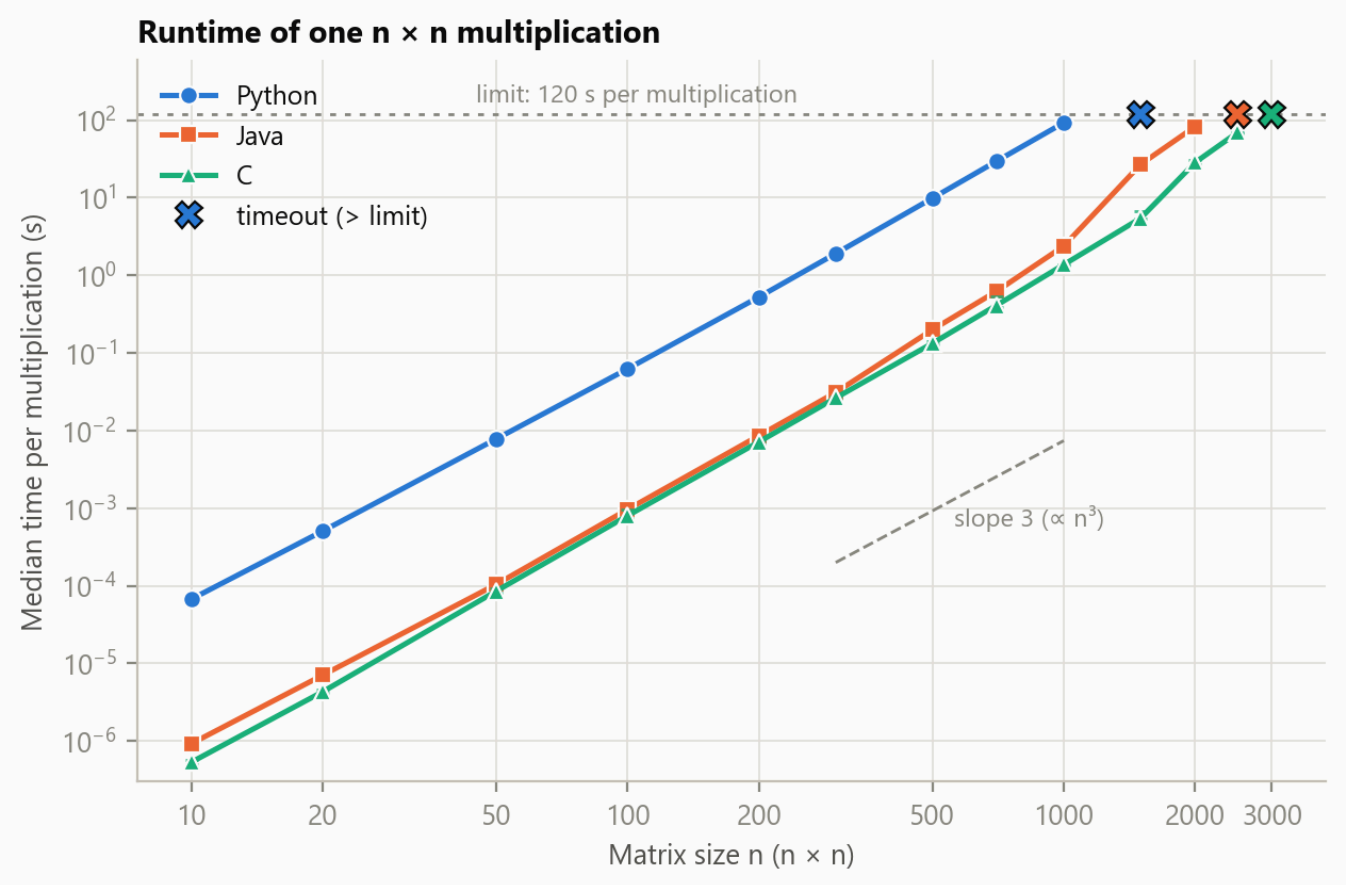


Figure 1. Median time of one multiplication against matrix size (log-log). Vertical bars are the IQR; × marks a launch stopped at the per-multiplication limit. The dashed line only shows the slope of n^3; its position is arbitrary.

Table 5. Median time per inner iteration, t/n^3 (ns). A constant would mean exactly cubic growth.

n	Python	Java	C
10	67.61	0.93	0.54
20	63.58	0.89	0.54
50	62.57	0.84	0.69
100	62.46	0.97	0.80
200	65.74	1.08	0.89
300	70.60	1.15	0.98
500	78.88	1.60	1.07
700	85.55	1.80	1.19
1000	92.38	2.38	1.37
1500	–	7.94	1.61
2000	–	10.29	3.54
2500	–	–	4.40

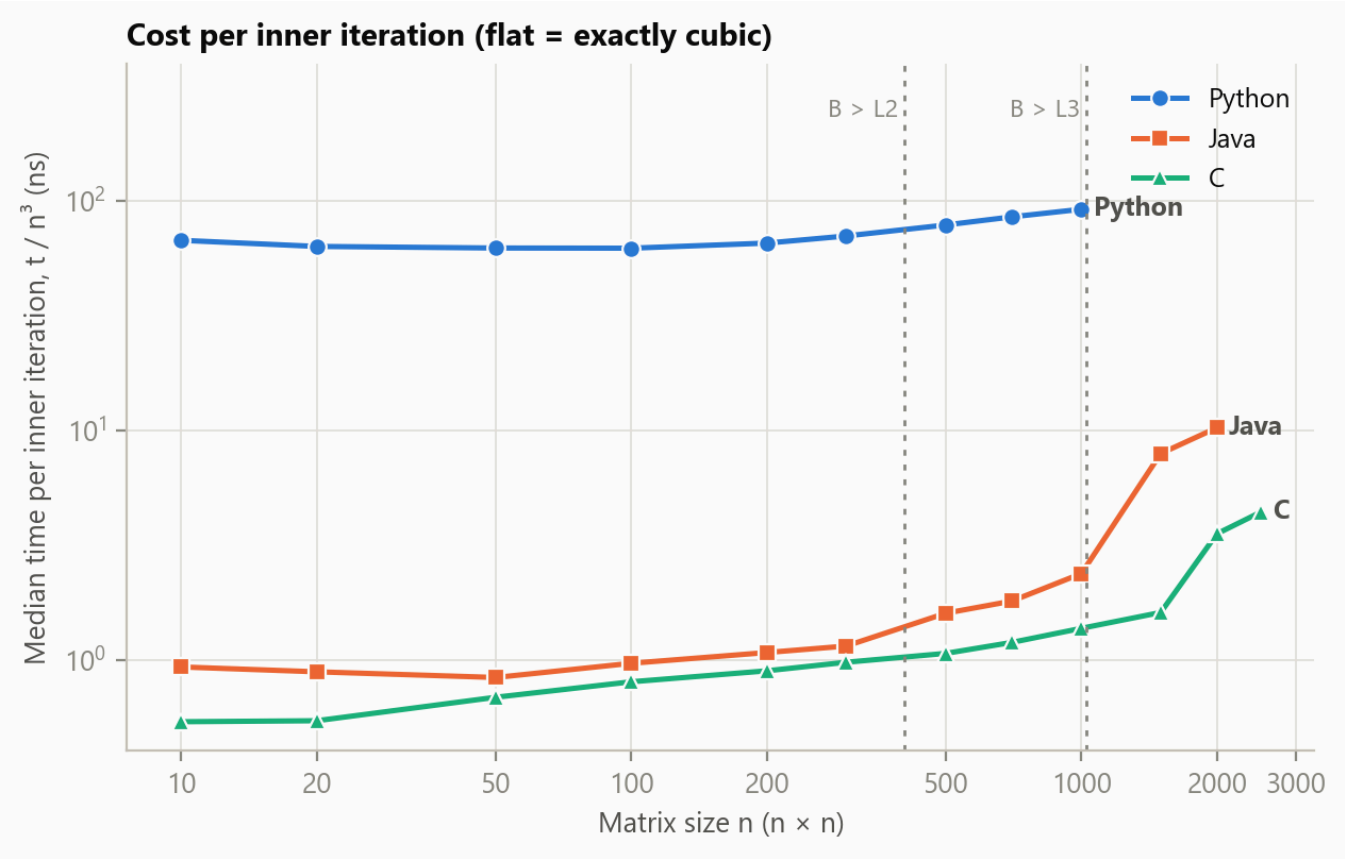


Figure 2. Median time per inner iteration, t/n^3. Dotted lines mark where one matrix B stops fitting in the L2 and L3 caches (computed from the cache sizes, not measured).

Cost per operation. If the cost were exactly cubic, t/n^3 (Table 5, Figure 2) would be constant. It is nearly constant only for small sizes: Python stays at 63–66 ns up to n = 200 and C moves from 0.54 to 0.89 ns, Java from 0.84 to 1.08 ns for n = 50–200. After that it rises in every language: Python reaches 92 ns at n = 1000 (48 % above n = 100); C goes through 1.37 ns (n = 1000) and 1.61 ns (1500) to 3.54 ns (2000) and 4.40 ns (2500); Java through 2.38 ns, 7.9 ns and 10.3 ns. So the growth is cubic only approximately: close to cubic for small n, and clearly faster than cubic once the matrices are large.

Relative performance. Table 6 gives the ratios between languages. Python is 126× slower than C at n = 10, where fixed overheads weigh most, and 67–78× (median 73×) for $n \geq 100$. Java is 1.18–1.20× slower than C for n = 100–300, 1.5–1.7× for n = 500–1000, and 4.9× and 2.9× at 1500 and 2000: after warm-up the JIT-compiled Java loop is

close to C for small and medium sizes, but the gap grows, it does not close, at large sizes. The Java value at n = 1500 rests on one launch (the second launch failed, Section 4.4); the one measured batch of the failed launch, 26.3 s, is within 2 % of it.

Table 6. Ratios of median times (where both runs completed).

n	Python ÷ C	Java ÷ C	Python ÷ Java
10	126.0×	1.73×	72.7×
20	117.3×	1.64×	71.8×
50	91.2×	1.22×	74.8×
100	77.9×	1.20×	64.7×
200	73.5×	1.20×	61.1×
300	72.4×	1.18×	61.5×
500	74.0×	1.50×	49.4×
700	71.9×	1.51×	47.5×
1000	67.4×	1.73×	38.9×
1500	–	4.94×	–
2000	–	2.90×	–
2500	–	–	–

4.3 Memory

Table 7 and Figure 3 show the peak working set. Each process has a fixed runtime baseline that dominates at small n: 4.3 MiB for C, 10.8 MiB for Python and 41.8 MiB for Java (the JVM is the heaviest). On top of it, the data grow with n^2: at n = 1000 the growth above the baseline is 22 MiB in C, 24 MiB in Java and 116 MiB in Python, while the three matrices of 8-byte values alone need 22.9 MiB. C and Java stay close to that array size; Python needs about 5 times more because every element is a pointer to a separate `float` object. As a consequence Python's total passes Java's between n = 500 (40 vs 48 MiB) and n = 700 (67 vs 54 MiB), although Java starts 31 MiB higher. At n = 1500 and 2000 Java's peak is the same, 136 MiB, while the array size grows from 51 to 92 MiB: its memory does not track the data exactly (probably because the JVM grows its heap in chunks; I did not investigate).

Table 7. Peak working set of the process (MiB): median over launches (min–max when more than one launch). Last column: size of the three matrices at 8 bytes per element.

n	Python	Java	C	$24n^2$ bytes
10	10.8 (10.8–10.9)	41.8 (41.7–41.8)	4.3 (4.3–4.3)	0.00
20	10.9 (10.9–10.9)	42.1 (41.9–42.2)	4.3 (4.3–4.3)	0.01
50	11.1 (11.1–11.2)	42.1 (42.1–43.0)	4.3 (4.3–4.3)	0.06
100	12.0 (12.0–12.0)	42.1 (42.0–42.1)	4.3 (4.3–4.3)	0.23
200	15.6 (15.5–15.6)	42.7 (42.7–42.8)	5.0 (5.0–5.0)	0.92
300	21.5 (21.5–21.5)	45.1 (44.2–45.1)	6.1 (6.1–6.1)	2.06
500	40.0 (39.9–40.0)	48.2 (48.0–48.2)	9.6 (9.5–9.6)	5.72
700	67.4	53.8 (53.8–53.8)	15.0 (15.0–15.0)	11.22
1000	126.9	65.7 (65.6–65.8)	26.7 (26.7–26.7)	22.89
1500	–	136.4	55.3 (55.3–55.3)	51.50
2000	–	136.2	95.4 (95.4–95.4)	91.55
2500	–	–	147.0	143.05

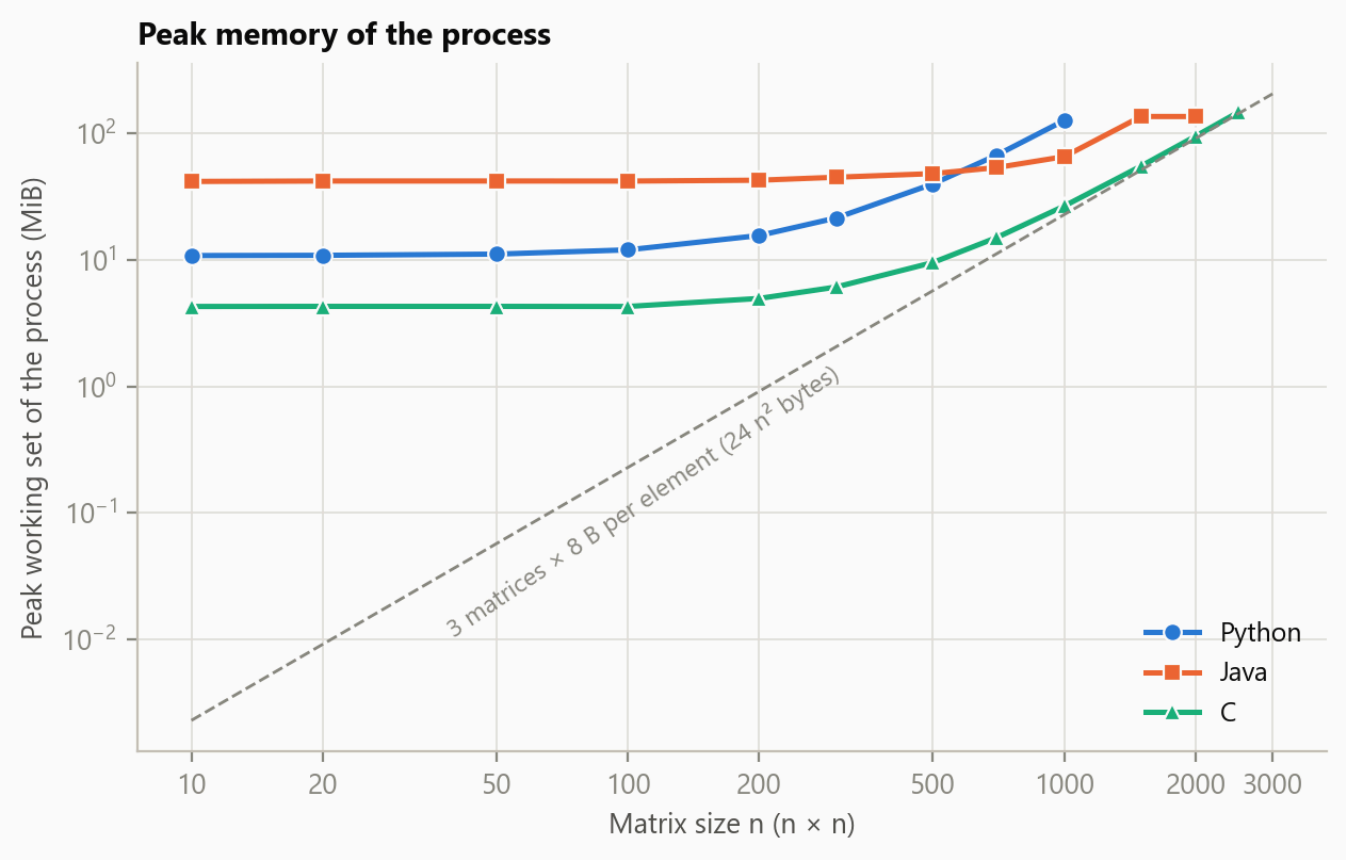


Figure 3. Peak working set of the process against matrix size (log-log).

4.4 Limits, timeouts and failures

The time budget was the only limit that bound. The largest size that met it was n = 1000 for Python (92.4 s per multiplication), 2000 for Java (82.3 s) and 2500 for C (68.8 s). One larger size was attempted per language (1500, 2500 and 3000); in each case one multiplication ran past 120 s and the launch was stopped (Table 9). These are timeouts at the limit; their real times are unknown. No launch came close to the 4 GiB memory budget (largest peak 226 MiB), so there were no memory failures. A fourth launch also failed: launch 61 (second launch of Java at n = 1500) completed two batches and then produced no output for more than 120 s; it was stopped after 824 s of wall time. I do not know why. Its batches stay in the raw file but are excluded from every statistic, so Java at n = 1500 has a single launch.

Table 8. Feasibility limits of the main comparison.

Language	Largest size meeting all limits	Largest size attempted	Limit per multiplication
Python	1000	1500	120 s
Java	2000	2500	120 s
C	2500	3000	120 s

Table 9. Launches that did not complete: stopped because a single multiplication exceeded the limit (timeout is a result, not an omission).

Language	n	Outcome	Wall time until stopped	Batches completed before
Python	1500	timeout_multiplication	121 s	0
Java	2500	timeout_multiplication	121 s	0
C	3000	timeout_multiplication	120 s	0
Java	1500	timeout_multiplication	824 s	2

4.5 Warm-up and variability

Warm-up matters mainly for Java (Figure 4): its first batch is 23× slower than the measured median at n = 10, 3.4× at n = 100 and about 1× from $n \approx 300$ on, which is why the calibration and warm-up batches are excluded and 5 warm-up batches are used for small sizes. For Python and C the first batch is within about ±30 % of the median at every size. Times also drift inside a launch: at n = 100 the C batches of the first launch end about 46 % slower than they start, probably a frequency or thermal effect that I did not measure; pooling launches that ran at different times averages over it.

The IQR is 10 % of the median for the typical configuration (range 0.2–36 %; Appendix A). The widest ranges come from two launches that started while the machine was busy with something else: the C launch at n = 700 was 51 % slower than the other launches of that configuration; the C launch at n = 1500 was 31 % slower than the other launches of that configuration. They were kept (no observation was removed); the medians stay close to those of the other launches because those outnumber them.

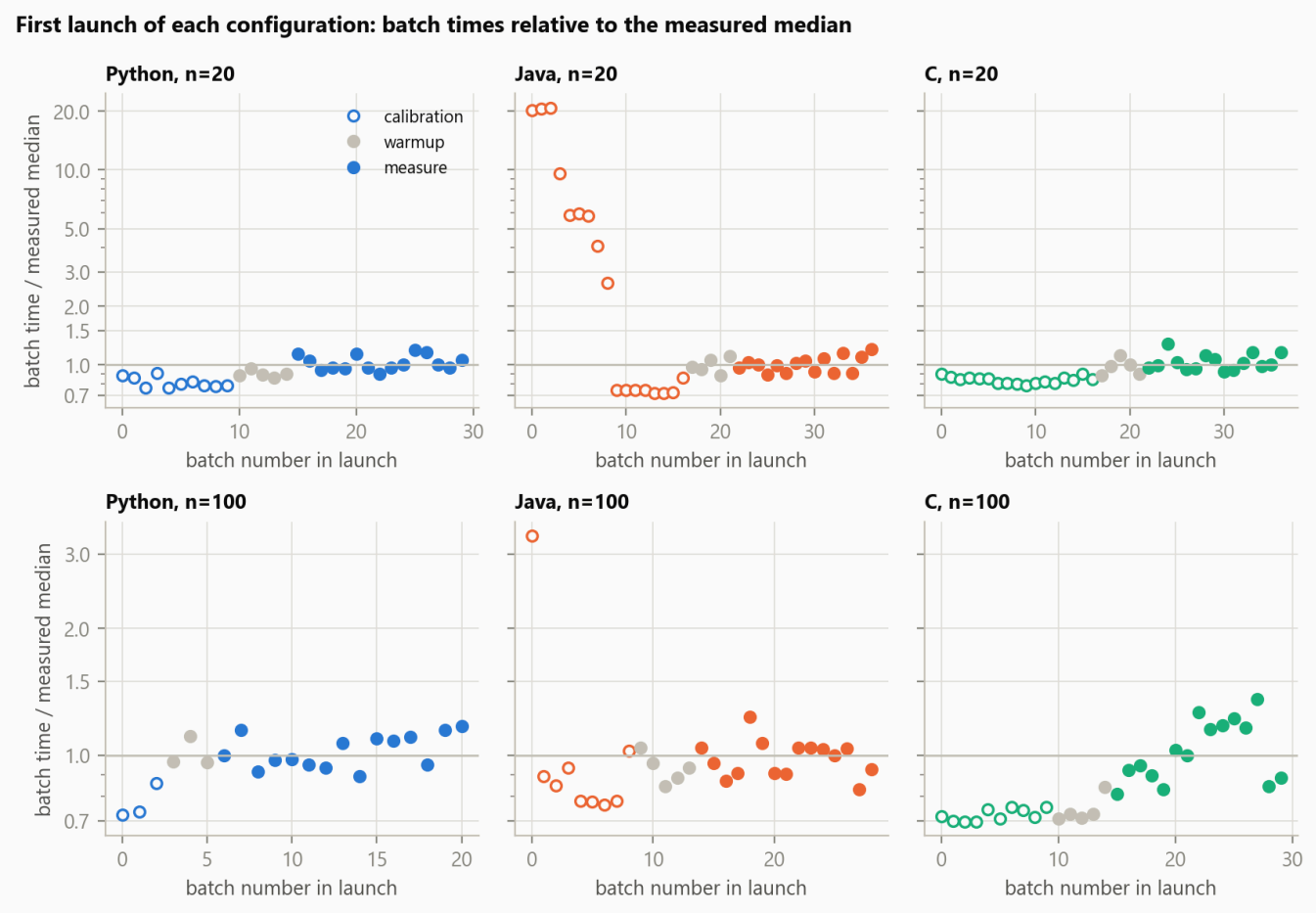


Figure 4. Batch times of the first launch of selected configurations, relative to that launch's measured median (log scale): open circles = calibration batches, grey = warm-up batches, filled = measured batches.

5. Discussion

Answer to the research question. Within the tested range, the time of one multiplication grows close to n^3 for small sizes and faster than that for large sizes, in all three languages (E1 only partly holds). The language mainly changes the constant: Python is about 73× slower than C, and Java 1.2–1.7× slower than C up to n = 1000 (E2 holds there), but 4.9× and 2.9× slower at n = 1500 and 2000. A statement that Java "approaches" C at large sizes would contradict these measurements. Memory has a fixed runtime baseline plus growth with n^2, and the languages differ in the per-element cost, not in the shape of the growth (E4).

Possible explanation of the rise in t/n^3 (a hypothesis, not tested). In the *i-j-k* order the inner loop reads one element of B from a different row at every step, so it touches n cache lines per column [2]. B stops fitting in the L2 cache at $n \approx 400$ and in the L3 cache at $n \approx 1000$ (from the cache sizes), and the cost per operation rises in that

region and beyond, which is consistent with a cache effect (E3). But the steepest steps come later ($n \approx 1500$–2000 for C) or earlier ($n \approx 1000$–1500 for Java) than those capacities, and I had no hardware counters to identify the mechanism. One possible reason for Java's earlier and larger step is its `double[n][n]` layout, which adds a row-pointer access to every iteration, but that would need a separate experiment (a flat Java array, a C version with separate rows) that I did not run.

Other points. Java's steady state, which Table 4 reports, is not what a short program would see, because the first batches are up to 23× slower. The Python result is for the literal textbook loop on CPython 3.12.0; libraries such as NumPy would change the constant by orders of magnitude, but that is a different experiment.

6. Limitations

- **One machine and one set of tools:** a thin-and-light laptop CPU with turbo (HP 250 G8, i5-1135G7), Windows 11, CPython 3.12.0, OpenJDK 25.0.2 with default JIT/GC, GCC 16.2.0 at `-O2`. Other hardware, operating systems, compilers or runtimes may give other constants and other step positions.
- **Machine state not fully controlled:** AC power and the Balanced plan, but clock frequency and thermal state were not controlled or recorded and the machine was not completely idle; two launches were visibly disturbed (Section 4.5).
- **Few launches at large sizes:** the most expensive configurations have one launch (Python ≥ 700, Java ≥ 1500, C 2500), so launch-to-launch variation is unknown there.
- **Memory metric:** the peak working set includes the language runtime and depends on how each runtime grows its memory; it is not live data or array size.
- **Kernel time only, one input type:** start-up, input and output are excluded; inputs are uniform in [0, 1); only square matrices; no hardware counters, so explanations of the large-n behaviour are hypotheses.

7. Conclusions

- For the tested implementations, runtime grows roughly as n^3 for small sizes; the cost per operation then rises in all three languages, steeply in C and Java above $n \approx 1000$–1500.
- Python is about 73× slower than C for $n \geq 100$. Java is 1.2–1.7× slower than C up to $n = 1000$ and 4.9× slower at 1500 (single launch).
- Memory is a fixed runtime baseline (4, 42 and 11 MiB for C, Java and Python) plus growth with n^2; at $n = 1000$ Python needs about 5 times the array size, C and Java about once.
- **Unresolved:** why the steep step occurs where it does and why it is earlier and larger in Java; this would need layout variants and hardware counters. No general claim that one language is faster than another follows from this study.

8. Use of generative AI

In line with the course policy, the use of AI is disclosed here. **Tool:** Claude Code (Anthropic). **Purpose and affected parts:** planning the experimental design; writing the benchmark programs, the driver, the test, analysis and build scripts; debugging the toolchain set-up on Windows; drafting this report; checking that cited web sources exist. **What the AI did not do:** no number in this report was produced or estimated by the AI; every result comes from running the published code on the author's machine. The author is responsible for understanding the code, for the measurements and for the conclusions.

References and materials

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022 (matrix multiplication, $\Theta(n^3)$ for the direct method).
2. U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Nov. 21, 2007. [Online]. Available: https://akkadia.org/drepper/cpumemory.pdf (Sec. 6.2.1, cache access in matrix multiplication).

3. OpenJDK, “Code Tools: jmh – Java Microbenchmark Harness.” [Online]. Available: https://openjdk.org/projects/code-tools/jmh (accessed Oct. 17, 2026).
4. Free Software Foundation, “Options That Control Optimization,” *GCC manual*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html (accessed Oct. 17, 2026).
5. Python Software Foundation, “time — Time access and conversions,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/time.html (accessed Oct. 17, 2026).
6. Microsoft, “Acquiring high-resolution time stamps,” Win32 apps documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps (accessed Oct. 17, 2026).
7. Microsoft, “PROCESS_MEMORY_COUNTERS_EX structure (psapi.h),” Win32 apps documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/psapi/ns-psapi-process_memory_counters_ex (accessed Oct. 17, 2026).
8. Universidad de Las Palmas de Gran Canaria, Big Data course 2026–2027, *Individual Assignments: Matrix Multiplication* and *Benchmarking Tools* (course material, campus virtual).

Materials. Everything is in the repository, https://github.com/abdela758/bigdata-matmul, and in `Assignment01_Choukri_Mohamed_Materials.zip`: source code (`src/`), the declared protocol (`config/experiment.json`), the build, test, benchmark and analysis scripts (`scripts/`), raw measurements (`results/raw_samples.csv`: one row per timed batch including warm-up; `results/launches.csv`: one row per process with status and peak memory), the tables and figures, the environment record and the correctness results. The shared inputs are not stored (up to 144 MB) but are regenerated bit-for-bit by `scripts/generate_inputs.py` and verified against SHA-256 hashes. The `README.md` lists the exact build and run commands and the tool versions.

Appendix A. Full results of the main comparison

Table A1. Every tested configuration of the main comparison: time per multiplication (all measured batches pooled).

Lang.	n	Launches	Samples	Mults per batch	Median	Q1	Q3	IQR/med (%)	Min	Max
Python	10	3	45	4096	67.6 µs	63.3 µs	74.3 µs	16.2	55.4 µs	90.8 µs
Python	20	3	45	512	509 µs	484 µs	527 µs	8.3	449 µs	595 µs
Python	50	3	45	32	7.82 ms	7.16 ms	8.72 ms	20.0	6.01 ms	19.5 ms
Python	100	3	45	8	62.5 ms	58.7 ms	65.2 ms	10.3	54.1 ms	76.1 ms
Python	200	3	30	1	526 ms	507 ms	563 ms	10.7	475 ms	626 ms
Python	300	3	30	1	1.91 s	1.85 s	2 s	8.0	1.76 s	8.06 s
Python	500	2	14	1	9.86 s	9.63 s	10.1 s	4.7	9.39 s	11.2 s
Python	700	1	5	1	29.3 s	29.1 s	30 s	3.1	29 s	35.4 s
Python	1000	1	5	1	92.4 s	92.2 s	92.9 s	0.8	91.8 s	95 s
Java	10	3	45	524288	930 ns	869 ns	976 ns	11.5	822 ns	1.15 µs
Java	20	3	45	65536	7.09 µs	6.73 µs	7.74 µs	14.3	6.34 µs	9.21 µs
Java	50	3	45	4096	105 µs	102 µs	108 µs	6.0	95.6 µs	146 µs
Java	100	3	45	256	965 µs	935 µs	1.03 ms	10.0	870 µs	1.29 ms
Java	200	3	45	32	8.6 ms	8.35 ms	8.89 ms	6.3	7.97 ms	10.4 ms
Java	300	3	45	8	31 ms	30 ms	32.7 ms	8.7	27.7 ms	42.7 ms
Java	500	3	45	2	200 ms	189 ms	206 ms	8.6	183 ms	265 ms
Java	700	3	30	1	618 ms	589 ms	693 ms	16.7	548 ms	3.18 s
Java	1000	3	30	1	2.38 s	2.32 s	2.48 s	6.4	2.27 s	2.68 s
Java	1500	1	5	1	26.8 s	26.8 s	26.8 s	0.2	26.4 s	26.9 s
Java	2000	1	5	1	82.3 s	81.9 s	82.8 s	1.0	81.4 s	93 s
C	10	3	45	524288	537 ns	504 ns	565 ns	11.3	472 ns	842 ns
C	20	3	45	65536	4.34 µs	4.14 µs	4.59 µs	10.5	3.47 µs	5.69 µs
C	50	3	45	4096	85.8 µs	81.5 µs	92.1 µs	12.3	74.6 µs	271 µs
C	100	3	45	512	802 µs	757 µs	975 µs	27.1	717 µs	2.01 ms
C	200	3	45	32	7.16 ms	6.94 ms	7.62 ms	9.6	6.41 ms	11.5 ms
C	300	3	45	16	26.3 ms	25.3 ms	27.9 ms	9.7	23.3 ms	34.6 ms
C	500	3	45	2	133 ms	130 ms	141 ms	8.2	121 ms	200 ms
C	700	3	30	1	408 ms	387 ms	534 ms	36.0	341 ms	715 ms
C	1000	3	30	1	1.37 s	1.35 s	1.39 s	3.3	1.32 s	1.47 s
C	1500	3	21	1	5.43 s	5.34 s	6.64 s	24.0	5.26 s	11.5 s
C	2000	2	10	1	28.4 s	27.2 s	30.2 s	10.6	26.9 s	31 s
C	2500	1	5	1	68.8 s	68.4 s	68.9 s	0.6	68.1 s	69 s